

Deep Reinforcement Learning With Adaptive Enhanced Attention for CTSPs

Renyi Zhang , Xiangping Xu , Xinli Shi , Senior Member, IEEE, Jinde Cao , Fellow, IEEE, and Jie Gui , Senior Member, IEEE

Abstract—The colored traveling salesman problem (CTSP) and its extended variant, the cumulative capacitated colored traveling salesman problem (C^2 -CTSP), represent significant generalizations of the classical multiple TSP. By incorporating color constraints, capacity restrictions, and customized cost calculation mechanisms for different entities, these formulations demonstrate strong relevance to real-world applications. This article proposes an adaptive enhanced attention (AEA) model to address color-constrained traveling salesman problems based on a reinforcement learning framework. Our method introduces several novel techniques. First, we develop a dynamic color-aware decoding mechanism that systematically handles color constraints through bitwise operations and adaptive state transitions. Then, we establish a bi-objective optimization method integrating an adaptive penalty mechanism to balance cumulative cost minimization with constraint satisfaction. In addition, we design an enhanced attention unit (EAU) that accelerates strategy convergence and improves solution precision through gated activation and hierarchical feature propagation. Evaluations on CTSP and C^2 -CTSP benchmarks demonstrate that the AEA model outperforms existing neural combinatorial optimization solvers and several heuristic baselines. Generalization experiments reveal that the EAU module effectively accelerates convergence and enhances decoding robustness, showing promising applicability to other combinatorial optimization tasks.

Impact Statement—The CTSP and its extended C^2 -CTSP find extensive applications in practical scheduling scenarios such

as automated guided vehicle dispatching and waterjet cutting machine scheduling. However, existing methods exhibit limitations in handling color constraints and optimizing cumulative costs. This article proposes an AEA model, which significantly improves solution accuracy and efficiency through dynamic color-aware decoding, bi-objective optimization, and an EAU. Benchmark evaluations demonstrate that the AEA model outperforms existing neural combinatorial optimization solvers and heuristic approaches. Additionally, the EAU module can accelerate the convergence of other combinatorial optimization tasks, providing an efficient solution for multiconstraint scenarios.

Index Terms—Adaptive enhanced attention (AEA), colored traveling salesman problems (CTSP), constrained combinatorial optimization, dynamic color-aware decoding, reinforcement learning (RL).

I. INTRODUCTION

MULTIPLE traveling salesman problem [1], [2] fails to account for practical scenarios where cities exhibit heterogeneous accessibility constraints for visiting salesmen. To address this limitation, Li et al. [3] pioneered a chromatic matching relationship model by introducing the Colored traveling salesman problem (CTSP). In this innovative framework, each city is assigned one or multiple colors, with the constraint that only salesmen possessing matching colors can visit it exactly once. Subsequent research has identified three distinct CTSP variants: Radial CTSP [3], [4], [5], [6], Serial CTSP [7], [8], [9], and Generalized CTSP [10], [11]. Recently, it has been further extended to a constrained form with capacity and time window constraints [12]. The CTSP paradigm has demonstrated significant practical utility across diverse applications, including waterjet cutting machine scheduling [8] and large automated guided vehicle (AGV) dispatching in automated depots [13], particularly for managing complex operational constraints.

As an nondeterministic polynomial-time hard (NP-hard) combinatorial optimization problem (COP), CTSP becomes computationally intractable for large-scale instances. To overcome this challenge, researchers have developed various heuristic and metaheuristic approaches such as genetic algorithms (GAs) [3], variable neighborhood search (VNS) [10], and delaunay triangulation-enhanced variable neighborhood search (DVNS) [11], which provide near-optimal solutions within practical time constraints.

While CTSP primarily focuses on minimizing travel costs through salesmen-centric optimization, this conventional cost-minimization framework proves inadequate for modeling

Received 11 December 2025; revised 29 January 2026; accepted 9 March 2026. Date of publication 13 March 2026; date of current version 31 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62473098 and Grant 62203108, in part by the Australian Research Council under Grant DE250100961, and in part by the Southeast University Interdisciplinary Research Program for Young. This article was recommended for publication by Associate Editor Dongbin Zhao upon evaluation of the reviewers' comments. (Corresponding authors: Xiangping Xu; Xinli Shi.)

Renyi Zhang and Jie Gui are with the School of Cyber Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: 220235524@seu.edu.cn; guijie@seu.edu.cn).

Xiangping Xu is with the School of Mathematics, Hohai University, Nanjing 211100, China (e-mail: xpxu2025@hhu.edu.cn).

Xinli Shi is with the School of Cyber Science and Engineering, and the National Center for Applied Mathematics in Jiangsu, Southeast University, Nanjing 210096, China, and also with the School of Engineering, RMIT University, Melbourne, VIC 3001, Australia (e-mail: xinli_shi@seu.edu.cn).

Jinde Cao is with the School of Mathematics, Southeast University, Nanjing 210096, China (e-mail: jdcdo@seu.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TAI.2026.3673681>, provided by the authors.

Digital Object Identifier 10.1109/TAI.2026.3673681

critical real-world scenarios. Humanitarian aid operations and disaster management systems [14], [15], [16], [17], for instance, prioritize objectives like casualty reduction and accelerated relief distribution over mere cost optimization. Existing models further struggle with complex transportation constraints involving cumulative costs and capacity limitations. To bridge these gaps, Xu et al. [18] proposed the cumulative capacitated colored traveling salesman problem (C^2 -CTSP), introducing the first integrated model incorporating both cumulative cost objectives and capacity constraints into the CTSP framework. Unlike traditional CTSP that minimizes total travel distance, C^2 -CTSP adopts a customer-centric paradigm that minimizes the cumulative arrival time at all customers, which is particularly relevant for time-sensitive applications. Additionally, each salesman has a limited capacity, and the total demand served by each salesman cannot exceed this limit. This formulation makes C^2 -CTSP well-suited for humanitarian logistics and emergency response scenarios where rapid service delivery is prioritized over vehicle-centric cost optimization [18]. Their generalized variable neighborhood search (GVNS)-based metaheuristic demonstrated superior solution quality. However, such heuristic approaches remain constrained by their reliance on domain-specific knowledge for manual rule construction, which limits scalability. Moreover, their convergence properties depend heavily on initial solution quality and operator selection, potentially leading to computational inefficiency and local optima entrapment.

In recent years, the field of neural combinatorial optimization (NCO) has emerged as an effective paradigm for addressing COPs. By leveraging data-driven autonomous learning mechanisms, these approaches circumvent the manual rule design processes inherent in conventional methods. Under the reinforcement learning (RL) framework, NCO methodologies effectively balance the exploration-exploitation trade-off, enabling efficient search within complex solution spaces.

The pioneering work of Vinyals et al. [19] with Pointer Networks laid the foundation for integrating neural networks and machine learning techniques into COPs. Subsequent extensions by Bello et al. [20] and Nazari et al. [21] demonstrated near-optimal performance in vehicle routing problem (VRP) solutions. Algorithmic innovations have progressively advanced the field, including Transformer-based self-attentive encoder architecture [22], policy optimization with multiple optima (POMO) method [23], and symmetry-aware NCO (Sym-NCO) framework [24], etc. These breakthroughs have substantially enhanced both solution quality and computational efficiency in combinatorial optimization.

Recent advancements in neural solvers have achieved multidimensional progress. In the scalability aspect, researchers have proposed novel training strategies for large-scale problem instances [25], [26], [27], [28], [29], [30], [31]. Regarding generalization capability, several studies have focused on cross-domain and cross-scale model adaptability [32], [33], [34], [35], [36]. For robustness enhancement, significant improvements have been made in addressing dynamic environments and data noise [37], [38]. Some other works consider enhanced neural solvers with RL models for VRP [39], [40] and variants such as multiagent routing [41], [42], while recent studies have further

advanced deep reinforcement learning (DRL)-based methods for VRP variants with complex constraints, such as energy-efficient electric vehicle routing [43] and heterogeneous capacitated vehicle routing problems (CVRPs) with unloading time constraints [44]. Notably, emerging research trends are exploring the integration of large language models to optimize VRP solution pipelines [45], [46], marking a promising direction for future developments in this domain.

Recent advances in multiconstraint optimization [47] and multimodal path planning [48] have demonstrated the effectiveness of knowledge-guided strategies and diversity-preservation mechanisms, inspiring our approach to color-constrained routing problems. However, research remains relatively scarce in specific problems such as the CTSP with additional color constraints. While Zhu et al. [49] pioneered the first NCO solver for CTSP, and recent work has further advanced CTSP solving through a Mamba-like gated linear attention architecture with normalization [50], existing approaches still demonstrate notable performance gaps when compared with conventional heuristic algorithms. This limitation primarily stems from color constraints restricting the available nodes, which necessitates that solvers simultaneously maintain constraint compliance while pursuing global optimality in distance costs. Furthermore, no existing studies have developed specialized NCO solvers for C^2 -CTSP. Current NCO solvers predominantly focus on minimizing travel path costs as their primary objective function, largely neglecting the exploration of cumulative cost-based problem formulations. To address these challenges, we propose an adaptive enhanced attention (AEA) model that provides a unified neural framework for solving color-constrained traveling salesman problems. Our approach systematically tackles three interconnected aspects: the architectural foundation for enhanced feature learning, the constraint-handling mechanism for color-aware decoding, and the problem-specific adaptation for cumulative cost optimization. The primary contributions of this research can be summarized as follows.

- 1) *AEA Framework*: We propose an enhanced attention unit (EAU) that synergistically integrates linear transformations and gated activation mechanisms, combined with feedforward normalization layers. This architecture serves as the foundation of our approach, accelerating solver convergence while enhancing solution accuracy for COPs. Experimental validation further demonstrates the framework's generalizability across diverse routing domains including TSP, CVRP, and vehicle routing problem with time windows (VRPTW).
- 2) *Dynamic Color Constraint Handling*: Building upon the EAU architecture, we develop a dynamic color decoding mechanism specifically designed to address CTSPs. This mechanism integrates global color attribute embeddings during the encoding phase and incorporates a self-study module with reward normalization strategies during decoding. The bitwise operation-based constraint resolver dynamically enforces color feasibility through binary state transitions, enabling the model to effectively learn color-constrained routing distributions.
- 3) *First NCO Solver for C^2 -CTSP*: Leveraging the above architectural and constraint-handling innovations, we

present the first NCO solver for C²-CTSP that addresses the unique challenge of cumulative cost optimization. By innovatively redesigning the loss function, integrating an adaptive penalty mechanism, and proposing a bi-objective balancing framework, we effectively address the cumulative cost challenge and establish a new paradigm for customer-centric routing optimization.

The remainder of this article is structured as follows. Section II formally presents the CTSPs formulation and develops a DRL framework for solving CTSPs. The details of our proposed AEA are elucidated in Section III. Comprehensive experimental evaluations are demonstrated in Section IV. Finally, Section V concludes the work with discussions on future research directions.

II. PRELIMINARIES

In this section, we first formalize the CTSP and its extended variant C²-CTSP, and then present an RL model designed to solve these CTSPs.

A. CTSPs Description

1) *CTSP Formulation*: Consider a CTSP with n cities and m salesmen ($m < n$). The CTSP instance is defined over a graph $G = (V, E)$ with a salesman set $\langle m \rangle = \{1, \dots, m\}$. The set of nodes is $V = \{v_0, v_1, \dots, v_n\}$, and the directed edges between all nodes are given by $E = \{e(v_i, v_j) \mid v_i, v_j \in V, i \neq j\}$. Each salesman k is assigned a unique color c_k , where $k, c_k \in \{1, 2, \dots, m\}$. For each city $v_i \in V$, let $c(v_i)$ represent the set of colors associated with v_i , where $1 \leq |c(v_i)| \leq m$. If $c_k \in c(v_i)$, then salesman k is permitted to visit city v_i . All salesmen start their routes from a common depot v_0 , select cities they can visit based on their assigned colors, complete their tours, and return to v_0 .

Based on the definition provided above, we constructed a 0–1 integer programming model for the CTSP, based on the work [9]. The objective function can be expressed as follows:

$$f = \min \sum_{k=1}^m \sum_{i=0}^n \sum_{j=0}^n w_{ij} x_{ij}^k. \quad (1)$$

Here, $w_{ij} \geq 0$ represents the travel cost from city v_i to v_j , such as distance. The 0–1 decision variable x_{ij}^k indicates whether salesman k travels from city v_i to city v_j . If this is the case, $x_{ij}^k = 1$; otherwise, $x_{ij}^k = 0$. Our objective is to minimize the total travel costs for all salesmen while strictly adhering to the nodes' color constraints. The CTSP model satisfies the following constraints:

Constraint 1: Each salesman is required to start from the depot v_0 , visit a series of cities, and ultimately return to the depot v_0 .

$$\sum_i x_{0i}^k = 1, \sum_i x_{i0}^k = 1, \forall v_i \in V, \forall k \in \langle m \rangle. \quad (2)$$

Constraint 2: Each salesman k is restricted to visiting only cities that are colored with c_k

$$\sum_i \sum_j x_{ij}^k = 0$$

$$\sum_i \sum_j x_{ji}^k = 0, \forall i \in \langle n \rangle, \forall v_j \in V \setminus V_k, \forall k \in \langle m \rangle. \quad (3)$$

Constraint 3: Except for v_0 , each city can be visited only once

$$\sum_i \sum_k x_{ij}^k = 1, \sum_i \sum_k x_{jik} = 1, j \neq i, \forall v_i, v_j \in V \setminus \{v_0\}, \forall k \in \langle m \rangle. \quad (4)$$

Constraint 4: The traveling salesman routes must strictly eliminate any subtour formations. We introduce auxiliary variables $u_{ik} \in \{1, 2, \dots, n\}$ representing the visiting order of city v_i by salesman k , with $u_{0k} = 0$ for the depot. The subtour elimination constraint is

$$u_{ik} - u_{jk} + n \times x_{ij}^k \leq n - 1, \quad i \neq j, \forall v_i, v_j \in V, \forall k \in \langle m \rangle. \quad (5)$$

When $x_{ij}^k = 1$ (salesman k travels from v_i to v_j), this ensures $u_{jk} \geq u_{ik} + 1$, preventing the formation of disconnected subtours.

In other words, for a CTSP instance with m salesmen, the solution must contain exactly m Hamiltonian cycles.

2) *C²-CTSP Formulation*: C²-CTSP extends the CTSP by incorporating capacity constraints and introducing cumulative cost as an optimization objective [12]. This enhancement makes C²-CTSP more suitable for real-world applications that require rapid responses. Specifically, for a C²-CTSP instance with n cities and m salesmen ($m < n$), it is defined over a graph $G = (V, E)$, where the set of salesmen is $\langle m \rangle = \{1, \dots, m\}$. The node set V and edge set E are defined in the same manner as in CTSP. Each salesman k is assigned a unique color c_k and a maximum capacity Q_k . Additionally, each city v_i may carry one or more colors and has a demand q_i .

As an extended variant of CTSP, C²-CTSP inherits and applies Constraints 2 (color compliance) and 3 (single-visit requirement) from CTSP. However, it operates under an open-path paradigm where salesmen depart from a central depot but are not required to return to it. Then, Constraint 1 is modified as Constraint 1'

$$\sum_i x_{0i}^k = 1, \forall v_i \in V, \forall k \in \langle m \rangle. \quad (6)$$

Additionally, it is important to note that the objective functions of the two CTSPs differ. The C²-CTSP focuses on cumulative costs. That is, given a route $r_k = (v_0, v_{i_1}, \dots, v_{i_{|V_k|}})$ with $v_{i_j} \in V_k \subseteq V$, its cumulative cost can be mathematically expressed as follows:

$$g(r_k) = \sum_{j=1}^{|V_k|} \sum_{l=1}^j w_{i_{l-1}, i_l}. \quad (7)$$

In C²-CTSP, for each city v_i visited by salesman k , y_i^k is defined as the cumulative cost of salesman k at node i starting from the depot; otherwise, $y_i^k = 0$. Then, Constraint 4 in CTSP is replaced by Constraint 4': when city v_j is visited immediately after city v_i by salesman k , it requires that

$$y_i^k + w_{ij} - (1 - x_{ij}^k) \cdot M \leq y_j^k, \forall v_i, v_j \in V, k \in \langle m \rangle \quad (8)$$

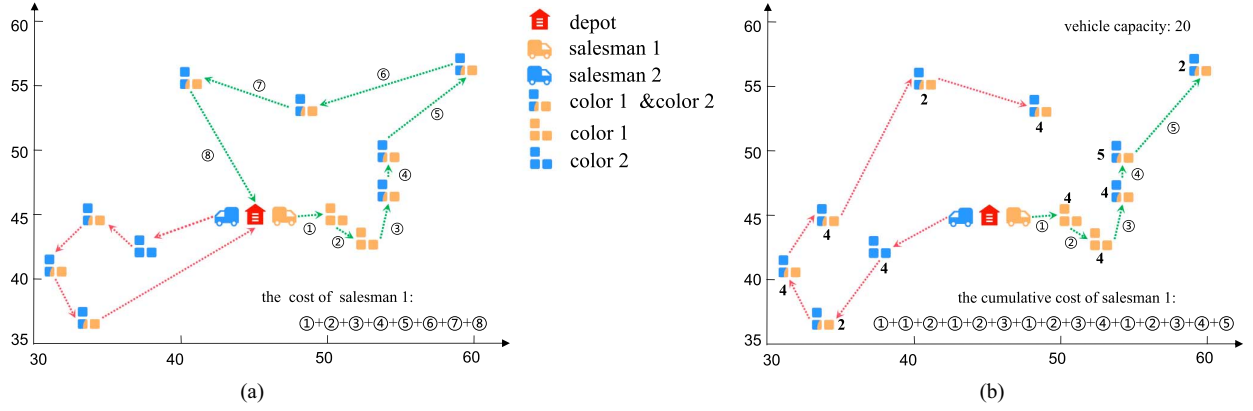


Fig. 1. Visualization of optimal routes for: (a) CTSP; and (b) C^2 -CTSP instances with two colors.

where $M > 0$ is a large positive number to eliminate subtours.

In addition, the incorporation of capacity constraints imposes additional requirements on C^2 -CTSP solutions, resulting in Constraint 5

$$\sum_{i \in V} \sum_{j \in V \setminus \{v_0\}} x_{ij}^k q_j \leq Q_k, \forall k \in \langle m \rangle. \quad (9)$$

Then, the objective function of C^2 -CTSP can be expressed as follows:

$$f' = \min \sum_{k \in \langle m \rangle} \sum_{i \in V \setminus \{0\}} y_i^k. \quad (10)$$

Regarding the fundamental distinction in optimization objectives, CTSP employs a salesman-centric paradigm that focuses on minimizing the total travel distance for all salesmen. In contrast, C^2 -CTSP adopts a customer-centric paradigm designed to minimize cumulative travel costs for all cities from the starting depot. To highlight the distinctive features of CTSP and C^2 -CTSP, we specifically illustrate the optimal routes for both problems in Fig. 1. It can be observed that the optimal routes for these two types of CTSPs differ significantly.

B. Reinforcement Learning for CTSPs

We begin by modeling the solution construction process of CTSPs as a constrained Markov decision process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$:

- 1) *State Space* $\mathcal{S}$: Encodes problem instances and partial solutions through feature vectors representing node coordinates, color assignments, available vehicle capacity, and route progressions.
- 2) *Action Space* $\mathcal{A}$: Comprises node selection operations where action $a_t = (v_i \rightarrow v_j)$ appends node v_j to the current partial route, subject to color constraints $c_k \in c(v_j)$.
- 3) *Reward Function* $\mathcal{R}$: According to the objective definition in problem modeling, as in CTSP, $R_t = -\|v_{t+1} - v_t\|_2$ provides stepwise negative Euclidean distance feedback to encourage path efficiency.
- 4) *Transition Dynamics* $\mathcal{P}$: The function $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ represents the transition probability function, which

defines the probability of moving from one state to another given a specific action.

The objective of the MDP is to learn a policy $\pi_\theta : \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A})$ that maximizes the cumulative reward over states while satisfying specific constraints. The policy π_θ is implemented as an encoder-decoder transformer network parameterized by θ . Given the MDP, we can define a parameterized policy $\pi_\theta(\delta|G)$ which takes a CTSP instance G as input and outputs a valid solution δ , where δ represents the complete solution (a set of m sub-tours). During training, we use τ to denote individual solution trajectories sampled from the policy, which include the sequential decision-making process and associated state transitions. Specifically, at each construction step, the policy $\pi_\theta(\delta|G)$ selects nodes either through *greedy selection* or *stochastic sampling*, progressively building a feasible color-constrained Hamiltonian cycle

$$\pi_\theta(\delta | \mathcal{G}) = \prod_{k=1}^m \prod_{t=1}^{|\delta_k|} \pi_\theta(\delta_{k,t} | \mathcal{G}, \delta_{k,t-1}) \quad (11)$$

where $\delta_{k,t}$ denotes the node selected by the k th salesman at the t th time step, i.e., the t th node in the sequence δ_k . Specifically, we use a single policy π_θ to sequentially construct m sub-tours (i.e., $\delta_1, \dots, \delta_m$) for each salesman, thereby forming a valid solution δ . This neural constructive heuristic allows us to learn specific routing patterns end-to-end while maintaining constraint satisfaction by masking the action space.

Remark 1: While Constraint 1' by (6) formally specifies only the departure requirement, the open-path property in C^2 -CTSP is enforced through the cumulative cost calculation mechanism rather than explicit return constraints. In the mathematical formulation, the objective function by (10) computes cumulative costs y_i^k only for customer nodes $i \in V \setminus \{0\}$, inherently excluding any return-to-depot costs. In our neural implementation, the autoregressive decoder generates a unified sequence where the depot node serves as a delimiter between different salesmen's routes. For instance, a decoded sequence $(v_0, v_1, v_2, v_3, v_0, v_4, v_5, v_6)$ is interpreted as two open paths: $r_1 = (v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3)$ for salesman 1 and $r_2 = (v_0 \rightarrow v_4 \rightarrow v_5 \rightarrow v_6)$ for salesman 2. When computing the

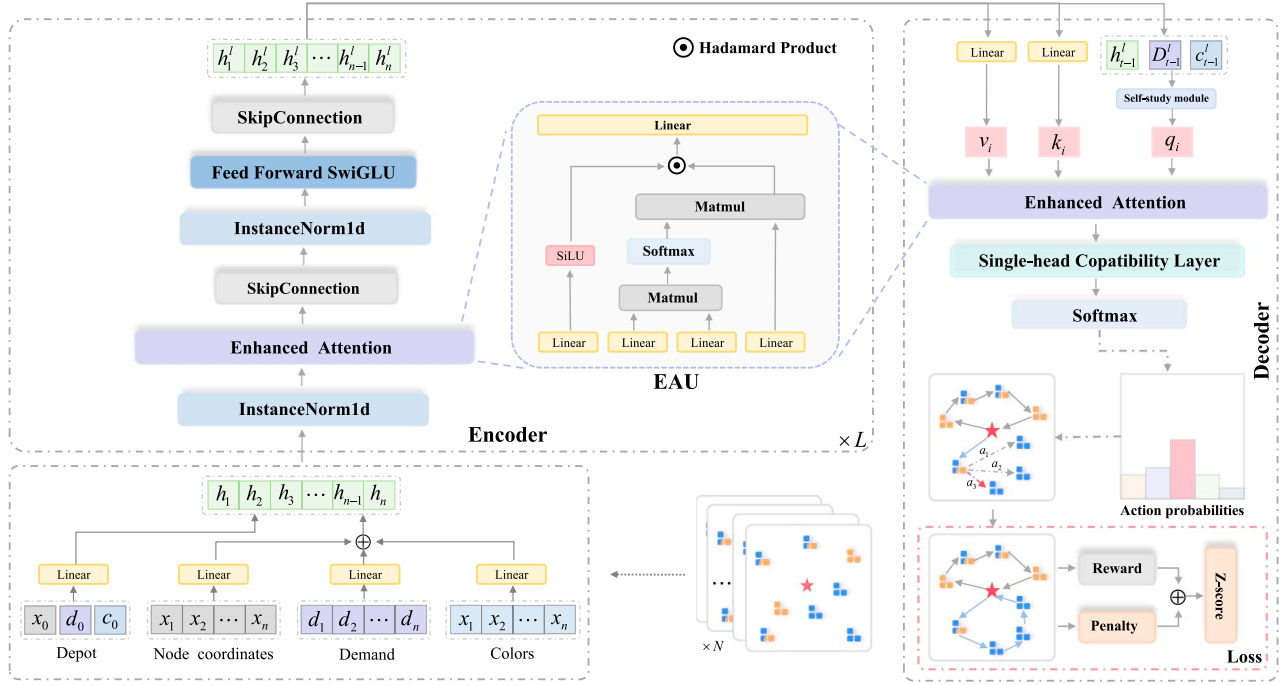


Fig. 2. AEA: Adaptive transformer framework for CTSPs. *Encoder (left)*: Node features (coordinates, demands, colors) are linearly transformed and aggregated into initial embeddings, then processed through L stacked layers of enhanced attention and feed forward SwiGLU with skip connections. The EAU module (center) applies SiLU-activated gating via Hadamard product to modulate attention outputs. *Decoder (right)*: Self-study module aggregates previous node embedding (h_{t-1}^l), remaining capacity (D_{t-1}^l), and color state (c_{t-1}^l) through learnable weights. Enhanced Attention computes compatibility scores, generating action probabilities via softmax. Training combines cumulative cost reward with adaptive penalty and Z-score normalization.

cumulative cost for training, we calculate:

- 1) For r_1 : $g(r_1) = d_{01} + (d_{01} + d_{12}) + (d_{01} + d_{12} + d_{23})$
- 2) For r_2 : $g(r_2) = d_{04} + (d_{04} + d_{45}) + (d_{04} + d_{45} + d_{56})$

where d_{ij} denotes the distance from v_i to v_j . The transition edges ($v_3 \rightarrow v_0$) and ($v_0 \rightarrow v_4$) are used solely for route segmentation and do not contribute to the cumulative cost. This design naturally realizes open-path routing through the loss function without requiring explicit architectural constraints in the decoder.

III. CONSTRUCTIVE MODEL FOR CTSPs

In this section, we present an AEA mechanism named by AEA, aiming to efficiently solve the CTSPs. Relying on the Transformer architecture, we develop an autoregressive neural solver based on a constructive approach. To enhance the model's representation ability of fine-grained features, we improve and add an EAU based on the standard softmax attention mechanism, thereby enhancing the network's capability to capture detailed features of COPs. Fig. 2 illustrates the complete architecture of the AEA and the information interaction mechanism among its core components.

The AEA framework comprises two core components: a feature encoding module and a decision decoding module. The encoding module employs a feature extraction network to map raw node features into a context-aware high-dimensional embedding space, effectively capturing topological correlations and dynamic constraints inherent in COPs. The decoding module leverages an attention-enhanced autoregressive mechanism to progressively construct feasible solution spaces that satisfy

problem constraints through MDP. In the next, the details of these two main components will be introduced.

A. Feature Encoding Module

The Feature Encoding Module establishes a unified representation space for heterogeneous node attributes in CTSPs. In the following, we thoroughly examine two fundamental modules integrated into the design, followed by an explanation of the encoder system's architecture.

1) *Global Color Attribute Embedding*: In contrast to the method proposed by Zhu et al. [49], the introduction of color properties as intrinsic parameters within the global encoding paradigm achieves enhanced feature representation. We represent a CTSPs instance as an aggregated tensor of all node features X , with a dimension of $(n+1)(2+M)$. Here, the $n+1$ dimension includes the depot node and n city nodes. The $2+M$ dimension is used to characterize the node features. Specifically, the "2" represents the two-dimensional spatial coordinates of the node, which clearly define its geometric position, and M corresponds to the node attributes (such as color attributes and capacity attributes).

For the node color attribute, we use binary encoding for abstract representation. For example, in a CTSP instance with four salesmen, the city color attribute code $[1, 1, 0, 1]$ indicates that all salesmen except the third one can access this city.

For the node features $X = \{x_0, x_1, x_2, \dots, x_n\}$, where each $x_i = (m_i, c_i, d_i)$ represents a tuple containing the spatial coordinates (m_i), color attribute (c_i), and demand capacity attribute (d_i) of the node. These input features are initially processed

through separate linear transformation layers to produce their corresponding feature representations

$$h_{m_i}^0 = W_m^0 n_i, h_{c_i}^0 = W_c^0 c_i, h_{d_i}^0 = W_d^0 d_i, \quad i = 1, \dots, n. \quad (12)$$

Here, W denotes the weight matrix. Specifically, for the distribution center node (denoted as node 0), its feature vector (m_0, c_0, d_0) follows the same transformation rules but utilizes distinct weight compared to regular nodes. The node features undergo linear transformation, and their hidden representations are further integrated to form the initial node embeddings, which can be expressed as follows:

$$h_i^0 = h_{m_i}^0 + h_{c_i}^0 + h_{d_i}^0 \quad i = 0, 1, \dots, n. \quad (13)$$

2) *EAU*: In (13), we obtain the initial node embeddings for each node. These embeddings are subsequently projected into distinct query (Q_i), key (K_i), and value (V_i) vectors through independent mapping functions. The compatibility score between node i and node j is computed using their respective query and key vectors. These compatibility scores are then normalized row-wise through a softmax function to derive the attention weights

$$Q_i = W_E^Q h_i^0, K_i = W_E^K h_i^0, V_i = W_E^V h_i^0 \quad (14)$$

$$\alpha_{i,j} = \text{Softmax} \left(\frac{Q_i^\top K_j}{\sqrt{d_k}} \right) \quad (15)$$

where d_k denotes the dimensionality of the key vector. The computed attention weights are then utilized to determine a weighted summation of value vectors (message aggregation mechanism), ultimately generating the attention output

$$A_i = \sum_{j=1}^n \alpha_{i,j} V_j = \frac{\sum_{j=1}^n S(Q_i, K_j) V_j}{\sum_{j=1}^n S(Q_i, K_j)}. \quad (16)$$

Standard softmax attention effectively captures pairwise node relationships but faces two limitations in color-constrained routing problems. First, the linear aggregation in (16) lacks sufficient capacity to model complex feasibility boundaries arising from dynamic color constraints, where the valid action space changes substantially across different states. Second, the uniform treatment of all attention features fails to account for the sparse, heterogeneous nature of color-constrained action spaces, in which only a subset of nodes may be accessible at each decision step. To address these limitations, we propose an EAU incorporating two key design principles.

a) *Nonlinear feature transformation*: As illustrated in Fig. 3, we introduce a nonlinear transformation layer using the SiLU activation function $\sigma(x) = x \cdot \text{sigmoid}(x)$. Unlike ReLU, which produces zero gradients for negative inputs, SiLU provides smooth, nonmonotonic gradients across its entire domain. This property is particularly important during the exploration phase: the nonzero gradients for negative values prevent the “dying neuron” problem and enable the model to learn from constraint violations. The transformation is expressed as follows:

$$Z_i = \text{SiLU}(W_z h_i^0) = W_z h_i^0 \cdot \text{sigmoid}(W_z h_i^0) \quad (17)$$

where W_z denotes the learnable weight matrix.

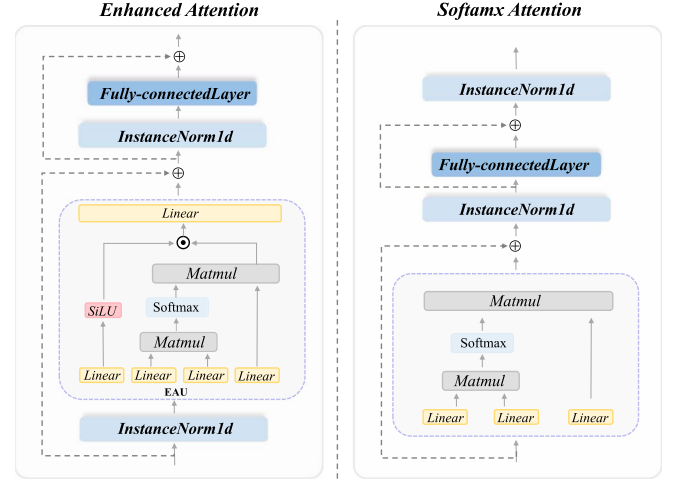


Fig. 3. Structural comparison between enhanced attention (left) and softmax attention (right). *Enhanced attention*: Embeddings are projected into Query, Key, Value vectors for scaled dot-product attention. The EAU module generates a SiLU-activated gating vector that modulates attention output via Hadamard product, followed by linear projection and normalization. *Softmax attention*: Standard architecture applies dot-product attention directly without gating modulation. The gating mechanism enables adaptive feature selection for constraint-relevant information.

b) *Multiplicative feature interaction*: We employ the Hadamard product to combine the nonlinear transformation Z_i with the attention output A_i

$$A_u = Z_i \odot A_i. \quad (18)$$

This multiplicative interaction serves as a gating mechanism enabling dynamic feature selection. The gating vector Z_i adaptively suppresses or amplifies different feature dimensions based on the current color state and constraint satisfaction requirements. Compared to concatenation-based methods requiring $O(d^2)$ complexity for additional projection layers, the Hadamard product operates in $O(d)$ complexity while effectively filtering irrelevant information from infeasible nodes.

Finally, we apply a linear transformation to project the gated features

$$\text{EAU}_i = W_h A_u. \quad (19)$$

In summary, the complete EAU module can be formulated as follows:

$$\begin{aligned} Z_i &= \text{SiLU}(W_z h_i^0) \\ A_u &= Z_i \odot A_i \\ \text{EAU}_i &= W_h A_u. \end{aligned} \quad (20)$$

This three-stage pipeline (activate $\rightarrow$ gate $\rightarrow$ project) implements selective attention refinement for constrained combinatorial optimization. The design preserves hierarchical propagation of node-wise relative positional encoding and topological relationships through cross-layer shortcut connections, facilitating gradient flow and enhancing feature discriminability. Empirical validation in Section IV-C demonstrates that each component contributes to both convergence speed and solution quality.

3) *Encoder Architecture*: During the encoding phase, we improve the standard attention model (AM) proposed by Kool et al. [22]. Specifically, first, we adopted the Feed Forward SwiGLU, which combines the gated linear unit (GLU) and the Swish activation function [51], to replace the feed-forward network based on the ReLU activation function in the original attention model. This effectively enhances the model's ability to capture complex features. Second, we introduced a normalization operation before the residual connection, changing the postnormalization structure to a prenormalization structure. This improvement strengthens the gradient flow and facilitates faster convergence of the network. Moreover, the effectiveness of this improvement has been verified in previous work [51]. The computational process of this module can be expressed as follows:

$$\begin{aligned}\hat{h}_i &= h_i^{(l-1)} + \text{EAU}(\text{BN}^l(h_i^{(l-1)}, \dots, h_n^{(l-1)})) \\ \mathcal{H}_i^{(l)} &= \hat{h}_i + \text{FF}(\text{BN}(\hat{h}_i)).\end{aligned}\quad (21)$$

The layer index l indicates a nonparameter-sharing configuration between layers. The EAU sublayer represents our novel attention mechanism, while the feed-forward (FF) sublayer integrates the key architectural concepts from both the Swish activation function and GLU.

B. Decision Decoding Module

The decision decoding module operates through three cascaded components: a self-study embedding constructor that generates context-aware representations via parameter modulation, an autoregressive probability estimator that establishes a Markov decision space using enhanced attention scores, and a bitwise operator-based constraint resolver that dynamically enforces color feasibility through binary state transitions. The details of these components are introduced below.

1) *Self-Study Module*: Without loss of generality, the autoregressive decoder progressively constructs valid solutions through sequential decision-making. At each time step t , the decoder selects a node based on prior information to iteratively build the solution. A notable innovation lies in our context embedding calculation process, where we introduce a Self-study Module that operates as follows:

$$e^t = \alpha f_h(h_{t-1}) + \beta f_d(d_{t-1}) + \gamma f_c(c_{t-1}). \quad (22)$$

Here, the function $f(\cdot)$ denotes a linear transformation, where h_{t-1} and d_{t-1} represent the embedding vector and capacity information of the previously selected node v_{t-1} , respectively. The variable c_{t-1} signifies the color state carried by the salesman at time t_{t-1} . We introduce three learnable parameters $\{\alpha, \beta, \gamma\}$ to model their dynamic relationships. The raw parameters $\{\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}\}$ are initialized uniformly as $\tilde{\alpha} = \tilde{\beta} = \tilde{\gamma} = 1.0$, ensuring equal initial weighting. To prevent uncontrolled growth and maintain numerical stability, we apply softmax normalization. To prevent uncontrolled growth of these parameters (α, β, γ) , we employ a softmax normalization function. This choice ensures that the parameter values remain stable during the learning process while preserving their relative significance in the decision-making mechanism.

2) *Autoregressive Decoding Module*: Next, we apply feed-back processing through the enhanced attention layer to refine the contextual embeddings e^t . This step derives attention weights and calculates similarity scores a_i between the query vector and all node embeddings ($\mathcal{H}_i = h_1, \dots, h_n$)

$$Q = W_D^Q e^t, K = W_D^K \mathcal{H}_i, V = W_D^V \mathcal{H}_i \quad (23)$$

$$Z = \text{SiLU}(W^Z \mathcal{H}_i) \quad (24)$$

$$a_i^t = \begin{cases} C \cdot \tanh\left(\frac{Q^\top K_i}{\sqrt{d}} \cdot Z\right), & v_i \text{ is valid} \\ -\infty, & \text{otherwise.} \end{cases} \quad (25)$$

Here, Z denotes the output vector from our enhancement module. Similar to the attention mechanism described in [16], our approach employs a tanh-based scoring mechanism. These computed scores are constrained within the range $[-C, C]$, where C is typically set to 10. This truncation is strategically applied to regulate numerical magnitudes.

Note that the accessibility of v_t is conditional, requiring $(\text{sum}(c_t \& c(v_i)) \neq 0) \wedge (v_t \neq v_{t'}, \forall t' < t) \wedge (d_t + d(v_i) \leq D)$ is satisfied. Finally, we apply the softmax function to derive the final probability distribution vector p through normalization

$$p_i^t = \frac{e^{a_i^t}}{\sum_{j=0}^n e^{a_j^t}}. \quad (26)$$

3) *Dynamic Color Decoding Mechanism*: The CTSPs involve route planning for m salesmen under dynamic color constraints. Let $\mathbf{1}_m$ and $\mathbf{0}_m$ be m -dimensional vectors with all ones and zeros, respectively. Each salesman depot from the warehouse carrying an initial color vector $c_0 = \mathbf{1}_m$ with the following operation rules:

a) *Accessibility constraint*: At any time t , the accessible city set is defined as follows:

$$V_t = \{v \in \text{unvisited cities} \mid c_0^t \wedge c(v) \neq \mathbf{0}_m\} \quad (27)$$

where $\wedge$ denotes the bitwise AND operation, and $c(v) \in \{0, 1\}^m$ represents the color attribute of city v .

b) *Color update rule*: When visiting a city v at time t , update

$$c_0^{t+1} = c_0^t \wedge c(v). \quad (28)$$

c) *Termination condition*: The termination protocol activates when it holds that

$$V_t = \emptyset \implies \text{return to depot at } t+1. \quad (29)$$

This ensures that all circuit cities satisfy

$$\bigcap_{v \in V_{\text{circuit}}} c(v) \neq \mathbf{0}_m. \quad (30)$$

This methodology leverages bitwise operations on color vectors to implicitly encode combinatorial constraints, thereby achieving a significant reduction in computational complexity during problem resolution (see Supplementary Section I and Section VI for detailed analysis). The dynamic interaction between the color vector's state transitions and constraint validation enables iterative bitwise AND operations to prune infeasible search paths while preserving solution feasibility.

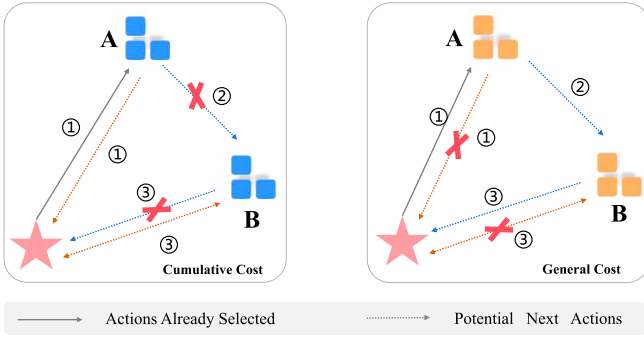


Fig. 4. Left panel illustrates the model's decoding preferences when using the cumulative cost formula as the loss function, while the right panel demonstrates the decoding tendencies observed when employing the standard cost coding formula as the loss function.

C. Training Module

The training module comprises three fundamental components: the technical design of loss functions tailored for C^2 -CTSP, the implementation of reward normalization mechanisms, and the establishment of a deep RL-based training paradigm.

1) *Cumulative Cost Loss Design:* As shown in Fig. 1, C^2 -CTSP adopts a cumulative cost calculation mechanism and does not need to consider the cost of the salesman returning to the depot. This difference makes it impossible for the C^2 -CTSP problem to directly use the cost calculation function in the CTSP problem as its model's loss function.

As illustrated in Fig. 4, when the model attempts to adopt the cumulative path cost as the loss function, it is observed that the network exhibits unexpected optimization behavior. Specifically, after choosing to visit node A from the depot node, the model is more inclined to generate the path sequence 'depot $\rightarrow$ A $\rightarrow$ depot $\rightarrow$ B' rather than 'depot $\rightarrow$ A $\rightarrow$ B'. This phenomenon can be explained by the triangle inequality: the cumulative path cost satisfies $\textcircled{1} + \textcircled{1} + \textcircled{2} > \textcircled{1} + \textcircled{3}$.

For a C^2 -CTSP instance with m salesmen and n cities, this design will lead the model to tend to generate n independent sub-paths during the training process instead of the required m sub-paths, thereby violating the C^2 -CTSP constraints established in Section II. Therefore, the C^2 -CTSP problem cannot directly follow the loss function design paradigm based on general cost in CTSP and needs to introduce specific constraints to ensure the correct mapping of the solution space.

To address the aforementioned challenges, we propose a restructured loss function incorporating an adaptive penalty mechanism

$$L = \mathcal{J}_{\text{cum}} + \lambda \mathcal{J}_{\text{penalty}}. \quad (31)$$

Here, $\mathcal{J}_{\text{cum}} = \sum_{j=1}^{|V_c|} \sum_{l=1}^j w_{i_{l-1}, i_l}$ defines the cumulative cost of solutions generated through autoregressive decoding, where w_{i_{l-1}, i_l} denotes the edge weight between nodes i_{l-1} and i_l in the construction sequence. To enforce constraints, we introduce a penalty term $\mathcal{J}_{\text{penalty}} = \xi \cdot \max_{\tau \in \mathcal{T}} \mathcal{J}_{\text{cum}}(\tau)$, where ξ serves as a hyperparameter controlling penalty intensity, and

$\mathcal{T}$ represents a set of n solution trajectories. These n trajectories are generated from the *same problem instance* by initiating the autoregressive decoder from n different starting nodes $\{v_1, v_2, \dots, v_n\}$. This multistart strategy enables diverse solution exploration within a single instance, where each trajectory τ_i represents a complete solution constructed starting from node v_i . Specifically, in instances involving m salesmen, this penalty term penalizes solutions where the number of subpaths in the solution space deviates from m . The dynamic coefficient $\lambda \in [0, 1]$ is defined as the proportion of constraint-violating instances within the current training batch.

This bi-objective balancing framework explicitly integrates the primary optimization target (minimizing cumulative path costs) with constraint satisfaction objectives (enforcing valid subpath configurations). The linear combination approximates Pareto optimality along the reward-plenty trade-off axis, thereby circumventing the computational complexity of conventional multiobjective optimization approaches.

The adaptive penalty mechanism enables violation-aware gradient regulation.

- 1) During low-violation phases ($\lambda \rightarrow 0$), optimization prioritizes cumulatively cost-efficient routing.
- 2) During high-violation regimes ($\lambda \rightarrow 1$), gradients emphasize constraint adherence.

Compared to static penalty formulations, this dynamic coefficient adjustment strategy eliminates the need for manual weight tuning, significantly reducing hyperparameter optimization overhead.

2) *Loss Normalization:* In the initial experimental phase, we observed that dynamic variations in the reward-penalty mechanism significantly affected the model's convergence rate. To enhance training stability and improve algorithmic performance, we implement a loss normalization technique. Specifically, prior to each model parameter update, we compute the mean (μ) and standard deviation (σ) of loss across all samples in the current batch

$$\mu = \frac{1}{B} \sum_{i=1}^B l_i, \quad \sigma = \sqrt{\frac{1}{B} \sum_{i=1}^B (l_i - \mu)^2} \quad (32)$$

$$A_i = \frac{l_i - \mu}{\sigma + \epsilon}. \quad (33)$$

Here, ϵ is a small positive constant (e.g., 10^{-8}) introduced to ensure numerical stability and prevent division by zero.

This normalization approach effectively mitigates training instability caused by reward scale fluctuations. By maintaining consistent gradient magnitudes across iterations, the method accelerates convergence and enhances final performance.

3) *Detailed Training Process:* The proposed model is implemented within the POMO framework for training. The process initiates by designating n distinct nodes $\{v_1, v_2, \dots, v_n\}$ as exploration starting points. Subsequently, the network samples n solution trajectories $\{\tau_1, \tau_2, \dots, \tau_n\}$ via Monte Carlo methods, where each trajectory τ_i is defined as a sequential decision-making path.

For each solution trajectory τ_i , the corresponding return (total reward) $R(\tau_i)$ is calculated. To maximize the expected return

J , we employ gradient ascent with estimated gradients derived from the sampled trajectories

$$\nabla_{\theta} J(\theta) \approx \frac{1}{n} \sum_{i=1}^n (R(\tau^i) - b^i(s)) \nabla_{\theta} \log p_{\theta}(\tau^i | s). \quad (34)$$

The baseline mechanism $b^j(s_i)$ relies on a greedy deployment strategy with shared baseline parameters. Specifically, this mechanism can be conceptualized as the average return of n distinct greedy roll-out trajectories generated from n different starting nodes. The complete training procedure follows the REINFORCE algorithm with baseline subtraction and is detailed in Supplementary Section II and Algorithm S1.

IV. EXPERIMENT

In this section, we will introduce the design of the dataset and validate the superior performance of AEA over baseline methods through numerous experiments. Specifically, we will address the following research questions (RQs):

- 1) *RQ1*: How effective is the AEA framework in solving the CTSP and C²-CTSP, and what performance does it deliver?
- 2) *RQ2*: Can the proposed enhanced attention mechanism demonstrate comparable adaptability to other routing problems?
- 3) *RQ3*: What performance improvements can be attributed to the novel architectural components within the AEA framework?
- 4) *RQ4*: How can the AEA framework be integrated with existing solvers to handle larger-scale instances that are more representative of real-world applications?

A. Design of Dataset

In this study, the vertex coordinates in set V for each CTSP instance are randomly generated within the interval $(0, 1)$. For C²-CTSP instances, coordinates are sampled from the $(0, 100)$ range to ensure consistency with baseline algorithms and maintain a uniform distribution across training and testing phases. Each vertex's color is encoded as a binary value.

Following the capacity constraint design principles outlined in [18], a positive integer value is first selected as the maximum capacity limit for each salesman or vehicle. Subsequently, demand values for cities are iteratively generated until the cumulative demand of vertices sharing the same color does not exceed this capacity constraint. Through this method, each city is assigned m demand values corresponding to m salesmen or vehicles. The final demand for a city is determined by selecting the maximum value among its m assigned demands.

B. Baseline

A comparative analysis is conducted between AEA and the following baseline models.

- 1) *Gurobi* [52]: A high-performance mathematical programming solver designed for linear and mixed-integer programming.

- 2) *OR Tools* [53]: An open-source toolkit developed by Google, aimed at solving a variety of COPs.
- 3) *COPT* [54]: An optimization solver developed by Cardinal Operations, supporting a wide range of optimization models.
- 4) *LKH-3* [55]: An advanced heuristic algorithm specifically tailored for solving the TSP and its variants.
- 5) *A Variety of Satisfactory Heuristic Algorithms*: Greedy Genetic Algorithm (GAG) [3], GA-2Opt [3], PLS [56], DVNS [11], and GVNS [18].
- 6) *GEIAM* [49]: The first neural solver designed for CTSP, introducing an entropy-insensitive attention mechanism and achieving excellent performance on CTSP instances.

All neural solvers are implemented with the following unified configuration: The Adam optimizer is employed with a learning rate of $\eta = 1 \times 10^{-4}$ and weight decay of $\omega = 1 \times 10^{-6}$. Node embeddings are initialized as 128-D vectors ($d = 128$). The AEA encoder is configured with $L = 6$ layers, each equipped with eight parallel attention heads to capture information from multiple independent subspaces. The penalty coefficient ξ in the loss function is empirically set to 10 based on preliminary ablation studies. Similarly, the clipping parameter C in (25) is set to 10. Detailed sensitivity analyses for both hyperparameters are provided in Supplementary Section VII. All training procedures are conducted under the PyTorch framework using a single NVIDIA GeForce RTX 3090 GPU (24 GB), while testing and comparative experiments involving traditional exact solvers and heuristic algorithms are implemented on a separate PC equipped with an NVIDIA GeForce RTX 4060 GPU (16 GB).

C. Experimental Results

1) *RQ1: Main Results*: Due to the limited literature on machine learning-based approaches for color-constrained TSP variants (e.g., CTSP/C²-CTSP), comparable neural network models remain scarce. To address this gap, we adapt the POMO algorithm to color-constrained scenarios, enabling its application to both CTSP and C²-CTSP, and systematically compare it with our proposed AEA. Additionally, we implement instance augmentation for enhanced inference through coordinate transformation techniques [23].

a) *CTSP results*: For the CTSP problem, we follow the experimental settings proposed by Zhu et al. [49], conducting a comprehensive evaluation on CTSP instances with node sizes of 20, 50, 70, 100, 150, and 200. Considering the computational complexity of the CTSP integer programming model, we impose time limits proportional to the problem size for heuristic algorithms to obtain approximate optimal solutions. Based on the experimental results from 100 randomly sampled instances (see the Supplementary Section III and Table S1), the solution costs achieved by the AEA framework are comparable to those of exact solvers, significantly outperforming traditional heuristic algorithms and existing NCO solvers. Notably, the superiority of AEA over the baseline POMO algorithm further validates the effectiveness of the enhanced attention mechanism.

b) C²-CTSP results: For the C²-CTSP problem, as shown in Supplementary Section IV and Table S2, we conduct systematic validation using two different settings of salesmen ($m = 3$ and $m = 4$) across four node sizes ($n = 20, 30, 50, 70$). The experimental results indicate that the proposed AEA method achieves performance comparable to advanced heuristic algorithms such as GVNS and PLS on small-scale instances ($n = 20, 30$), with an average gap of less than 0.75%, while demonstrating significant advantages over GAs. Furthermore, on larger-scale instances ($n = 50, 70$), AEA achieves an average improvement of 0.2% over the baseline POMO model, which underscores its superior exploration capability in complex scenarios.

Additionally, for both CTSP and C²-CTSP, AEA achieves significant speed advantages over traditional heuristic algorithms, primarily attributed to two operational mechanisms. First, during the training phase, the model systematically learns the distribution patterns of optimal solutions by processing extensive problem instances. This knowledge reuse enables the model to directly leverage pretrained expertise during inference, rapidly generating high-quality solutions while eliminating the need for iterative searches from scratch. Second, the integration with GPU-accelerated parallel computing architectures further optimizes the solving process, significantly enhancing overall computational performance. Consequently, our model is well-suited for applications in scenarios that require the rapid generation of high-quality solutions.

2) *RQ2: Extended Experiments:* To evaluate the generalization capability of the proposed EAU in combinatorial optimization, we conduct comparative experiments on TSP and CVRP instances. For each problem type, models were retrained from scratch using the same hyperparameter configuration to ensure a fair comparison of the architectural contributions. The experimental design included three configurations.

- 1) The traditional POMO baseline.
- 2) AEA integrated with EAU.
- 3) AEA without the EAU module for an ablation study.

As shown in Supplementary Section V and Fig. S2, the EAU module significantly accelerates the convergence rate of NCO solvers while improving solution accuracy across three routing problems. Particularly in complex constraint scenarios such as CVRP and C²-CTSP, the module demonstrates its effectiveness in enhancing both the efficiency and accuracy of the solutions. These results further confirm the powerful modeling capability of EAU in addressing multiconstraint optimization problems. In particular, even in the AEA ablation framework without the EAU module, it still outperformed the traditional POMO baseline in terms of convergence rate and exploration accuracy, highlighting the synergistic contributions of other architectural components.

To further validate the effectiveness of the EAU module on constraint-heavy problems, we conduct additional experiments on the VRPTW, which introduces temporal constraints requiring each customer to be served within a specified time interval. As shown in Table I, AEA consistently outperforms POMO on both $n = 50$ and $n = 100$ instances, and the entries in bold indicate the best results among learning-based methods.

TABLE I
COMPARATIVE RESULTS ON VRPTW BENCHMARK
INSTANCES

Methods	$n = 50$		$n = 100$	
	Cost	Gap (%)	Cost	Gap (%)
LKH-3	16.53	0.00	26.60	0.00
POMO	16.80	1.63	27.14	2.03
AEA	16.73	1.21	26.98	1.43

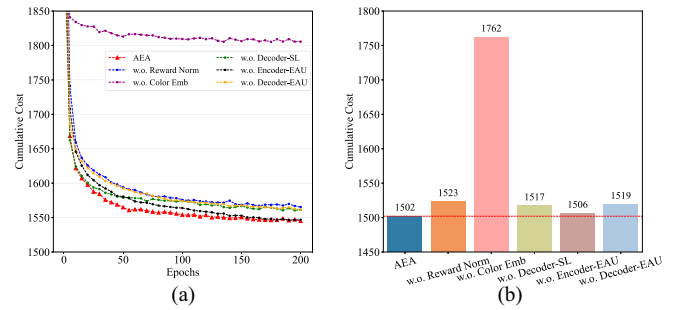


Fig. 5. Left panel (a) presents convergence curves from ablation studies over 200 training epochs, while the right panel (b) displays the corresponding average performance scores across 100 randomized test instances.

Specifically, AEA achieves optimality gaps of 1.21% and 1.43% compared to POMO's 1.63% and 2.03%, representing relative improvements of 25.8% and 29.6%, respectively. These results demonstrate that the EAU module effectively enhances the model's capability to capture complex constraint structures beyond color and capacity constraints, confirming its generalizability to other constraint-heavy COPs.

3) *RQ3: Ablation Studies:* We conducted an ablation study to evaluate the impact of newly introduced components on the AEA model using C²-CTSP instances with 20 nodes. Specifically, we removed the following components: 1) global color attribute embedding; 2) reward normalization; 3) decoder self-learning module; 4) encoder EAU module; and 5) decoder EAU module. The experimental results indicate that all of these components contribute to the performance of the AEA model.

In Fig. 5(a), we present a comparison of the model performance in different scenarios of removing components. Fig. 5(b) shows the solver's performance scores over 200 training epochs across 100 randomly generated instances. The results reveal that the absence of reward normalization has the most significant impact on the model's performance. Without normalization of rewards, the model is adversely affected by the varying scales of rewards and penalties, leading to reduced performance.

Particularly, we conduct separate ablation studies on the EAU modules in both the encoder and decoder. As illustrated in Fig. 5(a), the EAU in the decoder plays a critical role in enhancing model accuracy (yellow curve). While the absence of EAU in the encoder shows a relatively minor impact on final model precision, it significantly slows down the convergence speed (black curve). This observation suggests that in resource-constrained environments, removing the EAU unit from the

TABLE II
PERFORMANCE COMPARISON OF HYBRID AEA INTEGRATION
WITH HEURISTIC ALGORITHMS ON C²-CTSP ELI BENCHMARK
INSTANCES ($m = 4$)

Solvers	ELI101		ELI301	
	Gap (%)	Time (s)	Gap (%)	Time (s)
GVNS	0.00	1000	0.00	3000
DVNS	3.30	1000	8.47	3000
AEA+DVNS	2.83	1000	7.67	3000
GA	10.20	1000	30.28	3000
AEA+GA	7.67	1000	23.76	3000

encoder could be a viable strategy to conserve computational resources without significantly compromising the overall model performance.

The above structured analysis offers deeper insights into the importance of each component for improving the performance and efficiency of the proposed AEA model.

4) RQ4: Hybrid Integration for Larger Instances:

While NCO solvers demonstrate competitive performance on moderate-scale instances, their direct application to large-scale problems remains challenging due to computational constraints. To address this limitation, we investigate a hybrid integration strategy that leverages AEA-generated solutions as warm-start initializations for traditional heuristic algorithms.

a) Experimental setup: We evaluate this hybrid approach on C²-CTSP benchmarks. We employ the ELI benchmark instances (ELI101 and ELI301) with $m = 4$ salesmen. The AEA model is trained for 300 epochs, and the generated solutions serve as initial configurations for subsequent heuristic refinement.

b) Results on C²-CTSP: Table II presents results on ELI benchmark instances, where the bold entries indicate the results of the AEA-enhanced hybrid methods. The hybrid AEA+DVNS approach reduces the optimality gap from 3.30% to 2.83% on ELI101 and from 8.47% to 7.67% on ELI301, representing relative improvements of 14.2% and 9.4%, respectively. The improvement is more pronounced when combined with GA: AEA+GA achieves a gap reduction from 10.20% to 7.67% on ELI101 and from 30.28% to 23.76% on ELI301.

V. CONCLUSION

This article proposes an AEA model for solving color-constrained routing problems. First, a dynamic color decoding mechanism is developed to address the fundamental challenge of chromatic constraints in NCO methods. The architecture is further enhanced through two critical components: global color embedding for topological understanding of chromatic attributes and a reward normalization scheme for stable policy learning. Second, the first NCO solver is presented for C²-CTSP, where an adaptive penalty mechanism and dual-objective balancing framework are introduced to optimize cumulative cost objectives, demonstrating notable improvements over conventional approaches. Extensive experiments on CTSP and C²-CTSP benchmarks demonstrate that our method achieves

superior performance over existing neural NCO solvers and outperforms heuristic algorithms under time-constrained scenarios. Moreover, generalization experiments validate that the proposed EAU component enhances convergence efficiency and decoding quality across other routing problems.

While the proposed method achieves state-of-the-art neural solver performance, a slight optimality gap remains compared to expert-designed solvers such as LKH3. Additionally, the model requires retraining for different salesman configurations, and the quadratic attention complexity limits scalability for very large-scale instances. Therefore, future research can focus on developing dedicated constraint-handling networks and exploring multistage optimization frameworks to enhance the performance of neural architectures in multiconstraint scenarios. Potential real-world applications include automated warehouse logistics with capability-constrained AGVs, disaster relief operations with skill-matched teams, and healthcare routing with qualification-based personnel assignment.

REFERENCES

- [1] L. Tang, J. Liu, A. Rong, and Z. Yang, "A multiple traveling salesman problem model for hot rolling scheduling in Shanghai Baoshan iron & steel complex," *Eur. J. Oper. Res.*, vol. 124, no. 2, pp. 267–282, 2000.
- [2] T. Bektas, "The multiple traveling salesman problem: an overview of formulations and solution procedures," *Eur. J. Oper. Res.*, vol. 34, no. 3, pp. 209–219, 2006.
- [3] J. Li, M. Zhou, Q. Sun, X. Dai, and X. Yu, "Colored traveling salesman problem," *IEEE Trans. Cybern.*, vol. 45, no. 11, pp. 2390–2401, Nov. 2015.
- [4] V. Pandiri and A. Singh, "A swarm intelligence approach for the colored traveling salesman problem," *Appl. Intell.*, vol. 48, no. 11, pp. 4412–4428, 2018.
- [5] X. Dong, W. Dong, and Y. Cai, "Ant colony optimisation for coloured travelling salesman problem by multi-task learning," *IET Intell. Transp. Syst.*, vol. 12, no. 8, pp. 774–782, 2018.
- [6] P. He and J.-K. Hao, "Iterated two-phase local search for the colored traveling salesman problem," *Eng. Appl. Artif. Intell.*, vol. 97, 2021, Art. no. 104018.
- [7] J. Li, X. Meng, M. Zhou, and X. Dai, "A two-stage approach to path planning and collision avoidance of multibridge machining systems," *IEEE Trans. Syst. Man Cybern. Syst.*, vol. 47, no. 7, pp. 1039–1049, Jul. 2017.
- [8] J. Li, X. Meng, and X. Dai, "Collision-free scheduling of multi-bridge machining systems: a colored traveling salesman problem-based approach," *IEEE/CAA J. Autom. Sinica*, vol. 5, no. 1, pp. 139–147, Jan. 2018.
- [9] X. Meng, J. Li, M. Zhou, X. Dai, and J. Dou, "Population-based incremental learning algorithm for a serial colored traveling salesman problem," *IEEE Trans. Syst. Man Cybern. Syst.*, vol. 48, no. 2, pp. 277–288, Feb. 2016.
- [10] X. Meng, J. Li, X. Dai, and J. Dou, "Variable neighborhood search for a colored traveling salesman problem," *IEEE Trans. Intell. Transp. Syst.*, vol. 19, no. 4, pp. 1018–1026, Apr. 2018.
- [11] X. Xu, J. Li, and M. Zhou, "Delaunay-triangulation-based variable neighborhood search to solve large-scale general colored traveling salesman problems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 3, pp. 1583–1593, Mar. 2021.
- [12] X. Xu, X. Shi, J. Cao, and W. Huang, "Capacitated colored traveling salesman problem with time windows," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 8057–8068, 2025.
- [13] D. Wang, "Application of colored traveling salesman problems into scheduling and coordination of multiple material handling robots," M.S. thesis, Nanjing, China: School Automat., Southeast Univ., 2019.
- [14] N. Kapucu and V. Garayev, "Collaborative decision-making in emergency and disaster management," *Int. J. Public Admin.*, vol. 34, no. 6, pp. 366–375, 2011.
- [15] Q. Shi, W. Yang, Z.-M. Yang, and Q.-C. Zhao, "Relief materials vehicles planning in natural disasters," *IEEE/CAA J. Autom. Sinica*, vol. 5, no. 2, pp. 595–601, Mar. 2018.

- [16] A. Girma et al., "IoT-enabled autonomous system collaboration for disaster-area management," *IEEE/CAA J. Autom. Sinica*, vol. 7, no. 5, pp. 1249–1262, Sep. 2020.
- [17] P. D. Sentia, S. Abdul Shukor, A. N. A. Wahab, and M. Mukhtar, "Logistic distribution in humanitarian supply chain management: A thematic literature review and future research," *Ann. Oper. Res.*, vol. 323, no. 1, pp. 175–201, 2023.
- [18] X. Xu, J. Cao, X. Shi, and S. Gorbachev, "Cumulative capacitated colored traveling salesman problem," *IEEE Trans. Cybern.*, vol. 54, no. 8, pp. 4553–4566, Aug. 2024.
- [19] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," *Adv. Neural Inf. Process. Syst.*, vol. 28, pp. 2692–2700, Dec. 2015.
- [20] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," 2016, *arXiv:1611.09940*.
- [21] M. Nazari, A. Oroojlooy, L. Snyder, and M. Takác, "Reinforcement learning for solving the vehicle routing problem," *Adv. Neural Inf. Process. Syst.*, vol. 31, pp. 9839–9849, Dec. 2018.
- [22] W. Kool, H. Van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *Proc. Int. Conf. Learn. Represent.*, 2019.
- [23] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min, "POMO: Policy optimization with multiple optima for reinforcement learning," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 21188–21198, Dec. 2020.
- [24] M. Kim, J. Park, and J. Park, "Sym-NCO: Leveraging symmetricity for neural combinatorial optimization," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 1936–1949, Dec. 2022.
- [25] S. Li, Z. Yan, and C. Wu, "Learning to delegate for large-scale vehicle routing," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 26198–26211, Dec. 2021.
- [26] Z. Zong, H. Wang, J. Wang, M. Zheng, and Y. Li, "RBG: Hierarchically solving large-scale routing problems in logistic systems via reinforcement learning," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2022, pp. 4648–4658.
- [27] H. Cheng, H. Zheng, Y. Cong, W. Jiang, and S. Pu, "Select and optimize: Learning to solve large-scale TSP instances," in *Proc. Mach. Learn. Res. PMLR*, 2023, pp. 1219–1231.
- [28] Y. Jin, Y. Ding, X. Pan, K. He, L. Zhao, T. Qin, L. Song, and J. Bian, "Pointerformer: Deep reinforced multi-pointer transformer for the traveling salesman problem," *AAAI Conf. Artif. Intell.*, vol. 37, no. 7, 2023, pp. 8132–8140.
- [29] X. Pan, Y. Jin, Y. Ding, M. Feng, L. Zhao, L. Song, and J. Bian, "H-TSP: Hierarchically solving the large-scale traveling salesman problem," *AAAI Conf. Artif. Intell.*, vol. 37, no. 8, 2023, pp. 9345–9353.
- [30] Q. Hou, J. Yang, Y. Su, X. Wang, and Y. Deng, "Generalize learned heuristics to solve large-scale vehicle routing problems in real-time," in *Proc. Int. Conf. Learn. Represent.*, 2023.
- [31] H. Ye, J. Wang, H. Liang, Z. Cao, Y. Li, and F. Li, "GLOP: Learning global partition and local construction for solving large-scale routing problems in real-time," *AAAI Conf. Artif. Intell.*, vol. 38, no. 18, pp. 20284–20292, 2024.
- [32] Z. Zhang, Z. Zhang, X. Wang, and W. Zhu, "Learning to solve travelling salesman problem with hardness-adaptive curriculum," *AAAI Conf. Artif. Intell.*, vol. 36, no. 8, pp. 9136–9144, 2022.
- [33] J. Bi, Y. Ma, J. Wang, Z. Cao, J. Chen, Y. Sun, and Y. M. Chee, "Learning generalizable models for vehicle routing problems via knowledge distillation," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 31226–31238, 2022.
- [34] J. Son, M. Kim, H. Kim, and J. Park, "Meta-SAGE: Scale meta-learning scheduled adaptation with guided exploration for mitigating scale shift on combinatorial optimization," in *Proc. Int. Conf. Mach. Learn. PMLR*, 2023, pp. 32194–32210.
- [35] J. Zhou, Y. Wu, W. Song, Z. Cao, and J. Zhang, "Towards omni-generalizable neural methods for vehicle routing problems," in *Proc. Int. Conf. Mach. Learn. PMLR*, 2023, pp. 42769–42789.
- [36] C. Wang, Z. Yu, S. McAleer, T. Yu, and Y. Yang, "ASP: Learn a universal neural solver!" *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 6, pp. 4102–4114, Jun. 2024.
- [37] S. Geisler, J. Sommer, J. Schuchardt, A. Bojchevski, and S. Günemann, "Generalization of neural combinatorial solvers through the lens of adversarial robustness," in *Proc. Int. Conf. Learn. Represent.*, 2022.
- [38] H. Lu, Z. Li, R. Wang, Q. Ren, X. Li, M. Yuan, J. Zeng, X. Yang, and J. Yan, "Roco: A general framework for evaluating robustness of combinatorial optimization solvers on graphs," in *Proc. Int. Conf. Learn. Represent.*, 2023.
- [39] Y. Wang, Y.-H. Jia, W.-N. Chen, and Y. Mei, "Soft parameter sharing model for cross-problem generalization in vehicle routing problems," *IEEE Trans. Artif. Intell.*, vol. 7, no. 1, pp. 471–485, Jan. 2026.
- [40] L. Gao, R. Wang, Z. Jia, and C. Liu, "Multi-objective optimization for traveling salesman problem: A deep reinforcement learning algorithm via transfer learning," *IEEE Trans. Artif. Intell.*, vol. 6, no. 4, pp. 896–908, Apr. 2025.
- [41] G. Hu, "Scalable learning for multiagent route planning: Adapting to diverse task scales," *IEEE Trans. Artif. Intell.*, vol. 5, no. 10, pp. 4996–5011, Oct. 2024.
- [42] X.-C. Liao, X.-M. Hu, X.-L. Chen, Y. Mei, Y.-H. Jia, and W.-N. Chen, "Learning to communicate among agents for large-scale dynamic path planning with genetic programming hyper-heuristic," *IEEE Trans. Artif. Intell.*, vol. 6, no. 5, pp. 1269–1283, May 2025.
- [43] Q. Guan, H. Cao, J. Tan, L. Jia, D. Yan, and B. Chen, "Token-specific deep reinforcement learning for energy-efficient capacitated electric vehicle routing problems," *Appl. Energ.*, vol. 396, 2025, Art. no. 126314.
- [44] Q. Guan, S. Xue, J. Tan, L. Jia, H. Cao, and B. Chen, "Dynamic embedding-based deep reinforcement learning for heterogeneous capacitated VRPs with unloading time constraints," *Expert Syst. Appl.*, vol. 293, 2025, Art. no. 128660.
- [45] F. Berto, C. Hua, N. G. Zepeda, A. Hottung, N. Wouda, L. Lan, J. Park, K. Tierney, and J. Park, "RouteFinder: Towards foundation models for vehicle routing problems," *Trans. Mach. Learn. Res.*, vol. 2025, no. 9, pp. 1–19, 2025.
- [46] Z. Lin, Y. Wu, B. Zhou, Z. Cao, W. Song, Y. Zhang, and S. Jayavelu, "Cross-problem learning for solving vehicle routing problems," in *Proc. Int. Joint Conf. Artif. Intell.*, 2024, pp. 6958–6966.
- [47] W. Li, R. Wang, Y. Heng, T. Zhang, and L. Wang, "Knowledge-guided evolutionary optimization for large-scale air defense resource allocation," *IEEE Trans. Artif. Intell.*, vol. 5, no. 12, pp. 6267–6279, Dec. 2024.
- [48] X. Yao, W. Li, X. Pan, and R. Wang, "Multimodal multi-objective evolutionary algorithm for multiple path planning," *Comput. Ind. Eng.*, vol. 169, 2022, Art. no. 108145.
- [49] T. Zhu, X. Shi, X. Xu, and J. Cao, "Reinforcement learning for solving colored traveling salesman problems: An entropy-insensitive attention approach," *IEEE Trans. Artif. Intell.*, vol. 5, no. 12, pp. 6699–6708, Dec. 2024.
- [50] C. Wen, X. Xu, X. Shi, J. Cao, and L. Hua, "MGLAN: A Mamba-like gated linear attention with normalization for solving CTSP," *IEEE Trans. Artif. Intell.*, early access, 2025, doi: 10.1109/TAI.2025.3630620.
- [51] N. Shazeer, "Glu variants improve transformer," 2020, *arXiv:2002.05202*.
- [52] Gurobi Optimization, LLC, "Gurobi Optimizer Reference Manual," 2021. [Online]. Available: <https://www.gurobi.com>
- [53] V. Furnon and L. Perron, "Or-tools routing library," Google, 2023. Accessed: Mar. 22, 2026. [Online]. Available: <https://developers.google.com/optimization/routing>
- [54] D. Ge, Q. Huangfu, Z. Wang, J. Wu, and Y. Ye, "Cardinal optimizer (COPT) user guide," 2022, *arXiv:2208.14314*.
- [55] K. Helsgaun, *An Extension of the Lin-Kernighan-Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems*. Roskilde: Roskilde University, vol. 12, pp. 966–980, 2017.
- [56] X. Wang, T.-M. Choi, Z. Li, and S. Shao, "An effective local search algorithm for the multidepot cumulative capacitated vehicle routing problem," *IEEE Trans. Syst. Man Cybern. Syst.*, vol. 50, no. 12, pp. 4948–4958, Dec. 2020.